

Project 3 : Actor-Critic — Learning to Pick

*Student 1: Shibani Senthilbabu**Student 2: Seenivasa Ramasamy*

1 Policy Iteration (10 points)

We kicked things off with a classic Policy Iteration problem on a 3-state MDP before touching any neural networks. The setup: states $S = \{A, B, C\}$, actions $\{X, Y\}$, discount $\gamma = 0.9$, with reward matrix R and transition matrices P_X, P_Y given in the problem spec.

1.1 How It Works

We start with $\pi^{(0)}(s) = X$ everywhere and evaluate it by solving the Bellman equations:

$$V(s) = R(s, X) + \gamma \sum_{s'} P(s' | s, X) V(s') \quad (1)$$

This is just a 3×3 linear system. Once we have V^π , we compute Q -values for both actions in each state:

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^\pi(s') \quad (2)$$

and greedily update: $\pi_{\text{new}}(s) = \arg \max_a Q(s, a)$.

1.2 Result

Converged in **two iterations**. The takeaway that carries into everything else: a good policy always balances the immediate reward $R(s, a)$ against the long-run value of where you land. That exact tradeoff shows up again in every algorithm in this project — just with a neural network standing in for the lookup table.

2 Deep Reinforcement Learning

2.1 Policy Gradient Derivation (10 points)

The core problem: we want $\nabla_\theta J(\theta)$ where $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G_0]$, but the expectation is over trajectories whose distribution depends on θ . The log-derivative trick gets us out of this:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right] \quad (3)$$

The environment dynamics drop out because they have no θ dependence. The causality assumption lets us replace the full return with $G_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$.

This estimator is unbiased but very noisy. The fix is to subtract a baseline $b(s_t)$ — any function of state, not action — which reduces variance without touching the expected gradient:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \underbrace{(G_t - V_\phi(s_t))}_{A_t} \right] \quad (4)$$

$A_t > 0$ means the action was better than average; $A_t < 0$ means it was worse. That's the advantage function, and it's what every Actor-Critic method optimises.

2.2 REINFORCE (10 points)

REINFORCE is the simplest version: run a full episode, compute G_t for every step, take one gradient step. It works in theory but is painfully slow in practice for two reasons:

- **High variance** — G_t includes rewards from far-future steps that had nothing to do with a_t .
- **Sample inefficiency** — you have to wait for the whole episode to finish before learning anything.

Actor-Critic is what happens when you fix both.

2.3 Actor-Critic (10 points)

Split the work: the **Actor** π_θ picks actions, the **Critic** V_ϕ scores states. Instead of waiting for the episode to end, we bootstrap a TD advantage at every step:

$$A_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (5)$$

The combined training loss is:

$$\mathcal{L} = \underbrace{-\mathbb{E}[\log \pi_\theta(a_t|s_t) A_t]}_{\text{actor}} + 0.5 \underbrace{\mathbb{E}[(A_t)^2]}_{\text{critic}} - \beta \mathcal{H}[\pi_\theta] \quad (6)$$

The entropy term $\beta \mathcal{H}$ stops the policy from going deterministic too early.

2.3.1 Architecture

Both actor and critic use the same MLP structure — 3 hidden layers, 512 units each, ReLU — but with separate weights and separate Adam optimisers.

Table 1: A2C network layout for LunarLander-v3.

Network	Layers	Output
Actor	$8 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow 4$	Action logits
Critic	$8 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow 1$	Scalar $V(s)$

2.3.2 Things That Actually Mattered

A lot of the hyperparameter work was straightforward. A few things genuinely changed the outcome:

Weight init — the bug that killed every early run. The actor's output layer needs $\text{gain} = 0.01$ so initial action probs are near-uniform ($\approx 25\%$ each). We originally wrote:

```
for m in self.nn:
    if isinstance(m, nn.Linear):
        gain = 0.01 if m == self.nn[-1] else 1.0 # WRONG
        nn.init.orthogonal_(m.weight, gain=gain)
```

The `m == self.nn[-1]` comparison is unreliable in PyTorch — it silently evaluated to `False` every time, so every layer got $\text{gain} = 1.0$ and the policy immediately collapsed to one action. The fix: apply the override explicitly after the loop.

```
for m in self.nn:
    if isinstance(m, nn.Linear):
        nn.init.orthogonal_(m.weight, gain=1.0)
nn.init.orthogonal_(self.nn[-1].weight, gain=0.01) # CORRECT
```

Entropy annealing. We start $\beta = 0.15$ and decay by 0.9998 per episode (floor 0.01). This keeps exploration alive long enough to escape the hovering trap — the local optimum where the agent learns that hovering is safer than crashing but never discovers that landing scores +100.

Delayed LR decay. StepLR (half every 1000 episodes) only kicks in after episode 2000. Decaying the LR while the policy is still in deeply negative reward territory just slows learning without any benefit.

2.3.3 Training Hyperparameters

Table 2: A2C hyperparameters.

Parameter	Value
Hidden units / layers	512 / 3
Learning rate (Adam)	5×10^{-4}
LR decay	$\times 0.5$ every 1000 eps, after ep 2000
Discount γ	0.99
Entropy β (start)	0.15
Entropy decay	0.9998 / episode, floor 0.01
Gradient clip	0.5
Critic loss weight	0.5
Total episodes	8000

2.3.4 Training Story

Training went through three phases. The first 500 episodes were pure chaos — reward bounced between -400 (crash) and $+250$ (lucky landing). From episode 500 to around 3000 the agent found the hovering trap and got stuck: it learned that crashing costs -100 and decided the safest thing was to just hover in place. The high entropy kept it exploring anyway, and by episode 3100 it broke through to consistent positive rewards.

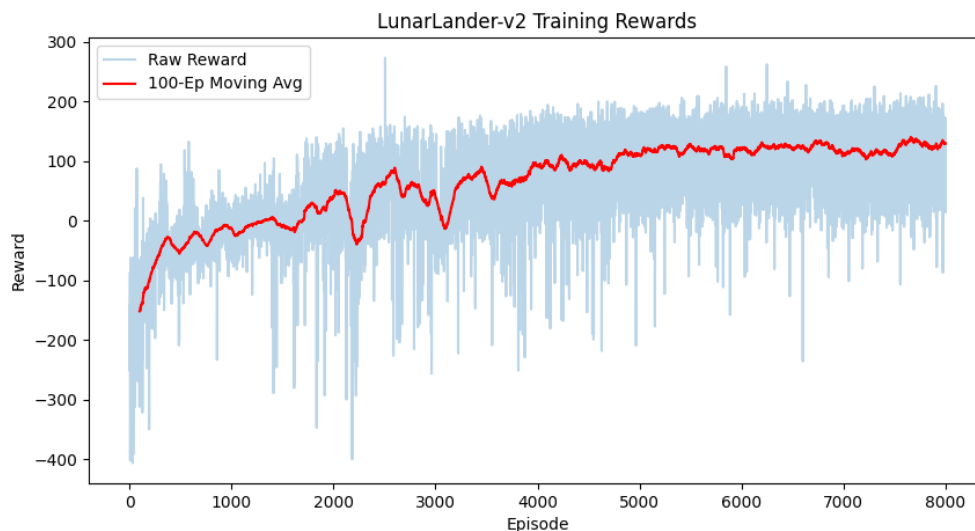


Figure 1: A2C reward history on LunarLander-v3. Early chaos, then the hovering plateau, then the breakthrough around ep 3100.

2.3.5 Final Results

```
python eval.py --checkpoint "./checkpoints/best_actor_critic.pt" --episodes 500
```

Table 3: A2C evaluation — 500 episodes, deterministic policy.

Metric	Value
Mean reward	+231.27
Std deviation	± 93.96
Peak episode	+320.03
Worst episode	−15.26
Project threshold	$> +100$

Mean of +231.27 — more than double the requirement. The ± 94 spread is just environment randomness (random spawn position and angle each episode), not policy inconsistency. Most episodes land in the +250–+320 range.

2.4 A3C: Asynchronous Advantage Actor-Critic (60 points)

The Kuka picking task gives the agent a camera image instead of a state vector, and the reward is a sparse binary (0 or 1 at episode end). That combination makes single-worker episodic training slow. A3C runs multiple workers in parallel — each with its own environment copy — and has them push gradients asynchronously to a shared global network. The diversity of worker experiences naturally decorrelates the data without needing a replay buffer.

2.4.1 Architecture

- **CNN:** $3 \rightarrow 32 \rightarrow 64 \rightarrow 64$ channels (kernel 3, stride 2, pad 1, ReLU) $\rightarrow$ AdaptiveAvgPool2d(2) $\rightarrow$ 256 features
- **Shared FC:** $256 \rightarrow 128 \rightarrow 64$ (ReLU)
- **Actor head:** $64 \rightarrow 64 \rightarrow 3$, Gaussian mean μ . Learned σ (init -1 , softplus ≈ 0.31), tanh-squashed
- **Critic head:** $64 \rightarrow 64 \rightarrow 1$

6 workers, $t_{\max} = 10$, SharedAdam, 15k episodes on Turing (≈ 1.5 hours).

2.4.2 What We Tried

Table 4: A3C tuning runs.

Run	Success%	Pool	σ_0	$t_{\max}$	γ	β
V1 (baseline)	44	Avg(1)	0	20	0.99	0.010
V2	43	Avg(1)	0	20	0.99	0.005
V3	0	None	0	20	0.99	0.010
V5	47	Avg(1)	−1	10	0.95	0.010
V6	52	Avg(2)	−1	10	0.95	0.005
V7	43	Avg(2)+BN	−1	5	0.95	0.005

The lessons, in short:

- $\sigma_0 = 0$ **is too exploratory.** softplus(0) ≈ 0.69 — way too much noise on a precision task. Worse, σ kept drifting *up* during training in V1/V2, meaning the policy was getting less confident over time. Setting $\sigma_0 = -1$ (softplus ≈ 0.31) fixed it.
- $t_{\max}$ **must be less than episode length.** With $t_{\max} = 20$ (= episode length), every rollout was a complete episode and the critic’s bootstrapping term never fired. It was just Monte Carlo. Halving to $t_{\max} = 10$ restored mid-episode TD updates.
- $\gamma = 0.99$ **does nothing on 20-step episodes.** $0.99^{20} \approx 0.82$ — barely any discounting. Dropping to $\gamma = 0.95$ ($0.95^{20} \approx 0.36$) gave the agent a real incentive to grasp early rather than late.
- **Pooling keeps spatial info.** AvgPool(1) collapses to 64 numbers and loses all object location info. AvgPool(2) keeps a 2×2 grid (256 features) and bumped success from 47% to 52%. No pooling at all (V3, 1600 features) caused actor loss to explode to ± 20000 .

- **BatchNorm breaks A3C.** Each worker sees different data, so batch stats diverge across workers. V7 dropped from 52% back to 43%.

2.4.3 Final Config

Table 5: Final A3C hyperparameters (V6).

Parameter	Value
Workers / rollout $t_{\max}$	6 / 10
Discount γ	0.95
Entropy β	0.005
Value loss coeff	0.5
Learning rate	2×10^{-4}
Grad clip	40.0
Optimizer	SharedAdam
σ init	-1.0
Image size / pooling	40×40 RGB / AdaptiveAvgPool2d(2)
Shared FC / head hidden	[128, 64] / [64]
Weight init	Xavier uniform
Episodes	15,000

2.4.4 Results

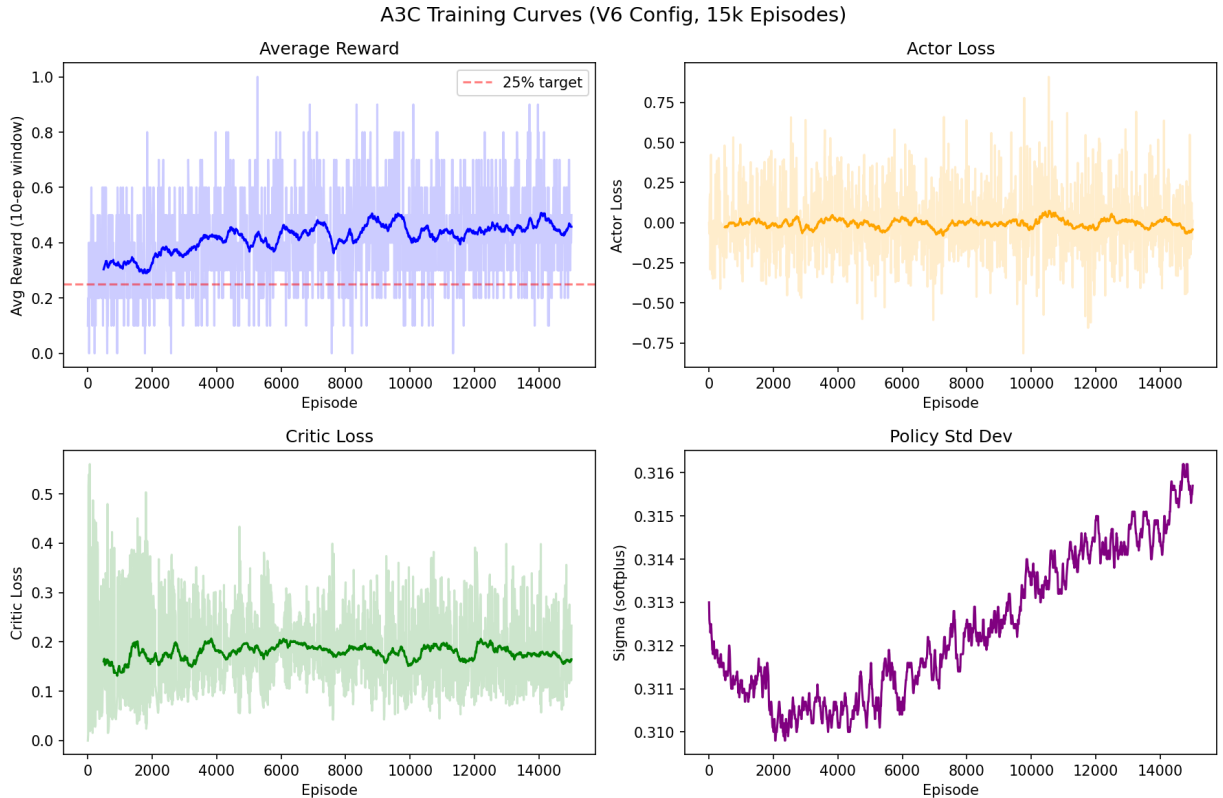


Figure 2: V6 training curves (15k eps, 6 workers). Reward stabilises at 0.4–0.5 early. σ barely moves after a few hundred episodes — the policy locked in an action scale and spent the rest of training refining aim.

Table 6: A3C evaluation — 100 episodes, deterministic policy.

Metric	Value
Success rate	52% (52/100)
Avg reward	0.52 ± 0.50
Avg episode length	7.02 ± 0.14 steps
Threshold	25%

52% against a 25% target. The ± 0.50 variance on a binary reward is the theoretical maximum — the agent either grabs the object or it doesn't, no in-between. Average episode length of 7 steps (out of 20 allowed) means it commits quickly and doesn't hesitate.

3 Conclusion

We covered the full arc from a 3-state MDP solved by hand to a CNN-based A3C agent picking objects out of a bin. The algorithms build on each other in a clear line: Policy Iteration is exact but needs a known model. REINFORCE drops that requirement but is noisy. Actor-Critic fixes the noise with bootstrapping. A3C scales everything up with parallel workers.

The results came out well. LunarLander hit +231.27 mean reward over 500 episodes (threshold: $> +100$). The Kuka picking agent hit 52% success over 100 episodes (threshold: $> 25\%$).

The more interesting story was in the debugging. The biggest single issue — a weight initialisation bug where `m == self.nn[-1]` silently failed and set the wrong gain on the output layer — killed every training run for a long time before we caught it. After that, the hovering trap on LunarLander (where the agent learns that hovering is safer than crashing but misses that landing scores +100) and the rollout length mistake in A3C (where $t_{\max} = T$ silently disabled TD bootstrapping) were the next big ones. None of these are obvious from reading the algorithm — they only show up when you look at the training curves and ask why nothing is improving.